Process and Decision Documentation

Group: 10B **Assignment:** A2 -- Mid-Term Game **Disability Topic:** Traumatic Brain Injury (TBI) **Date:** March 5, 2026

---

1. Role-Based Contributions

Member	Primary Role	Shadow Role	Key Contributions
Jimin Kim	Project Setup / Core Game Loop	Game Enhancement	Set up project structure and GitHub repository. Implemented core game state machine (start/play/win/lose). Later used Claude Code to enhance the full game with walls, rooms, stimulus zones, Web Audio API sound system, particle effects, vignette/shake effects, cognitive fatigue mechanic, and ACM references in code.
Kaiyang Sun	TBI Mechanics Design & Implementation	Gameplay Systems	Implemented Memory Fade and Sensory Overload mechanics, integrated them into the game loop, and added player movement and win/lose conditions.
Catarina Jin	Level Design	Gameplay Testing	Designed the playable level structure, positioned stars and the Calm Zone to guide player progression, added checkpoints and respawn logic, and conducted playtesting to identify areas where players became confused or stuck.
Annora Jo	UX/UI Design & Art/Sound	Documentation	Designed and polished the visual aspect of the game including player character facial expressions, Low Sensory Mode visual differences, overload meter display, and instruction screens. Enhanced accessibility design for both normal and Low Sensory modes.

---

2. Key Design Decisions

2.1 Project Structure

The project was organized following the assignment's required folder structure:

- `index.html` -- Main HTML file loading p5.js and the game sketch, with meta description and accessibility attributes
- `sketch.js` -- Primary JavaScript file containing all game logic (~1240 lines)
- `style.css` -- Stylesheet for page styling (centered canvas with drop shadow)
- `jsconfig.json` -- VS Code configuration
- `libraries/` -- Contains p5.min.js (v1.9.0) loaded locally

- `assets/` -- Contains `images/`, `sounds/`, and a README documenting that all assets are procedurally generated

2.2 Core Game Loop Architecture

The game uses a state machine pattern with four states: `STATE_START`, `STATE_PLAY`, `STATE_WIN`, and `STATE_LOSE`. The `draw()` function uses a `switch` statement to render the appropriate screen based on the current state. Each state has its own dedicated draw function. The `initLevel()` function resets all game variables when starting a new game.

2.3 Technology Choices

- **Framework:** p5.js (v1.9.0) -- required by the course
- **Audio:** Web Audio API (OscillatorNode + GainNode) for procedural sound generation
- **Canvas size:** 800 x 600 pixels
- **Hosting:** GitHub Pages, deployed from the `main` branch

2.4 Gameplay Mechanics Representing TBI

Three gameplay mechanics were implemented to represent aspects of living with Traumatic Brain Injury (TBI): **Memory Fade**, **Sensory Overload**, and **Cognitive Fatigue**.

The **Memory Fade** mechanic hides the level objective after a short period of time. Players can press the **M** key to briefly recall the objective. The recall duration becomes shorter in later phases, making memory management progressively harder. This mechanic was designed to simulate how TBI survivors may have difficulty maintaining task instructions or remembering goals during an activity (CDC, 2024; Johansson et al., 2009).

The **Sensory Overload** mechanic introduces an overload meter that gradually increases as the player moves through the level. **Stimulus zones** (red-tinted areas labeled "noise") increase the overload rate significantly faster. If the meter reaches its maximum value, the player either respawns at a checkpoint (if lives remain) or loses the game. A **Calm Zone** is placed in Room 3 where players can reduce overload by standing inside it. Visual effects including screen shake, tunnel vision vignette, and a white haze overlay activate at high overload levels. An ambient audio drone also increases in pitch and volume as overload rises, creating audio sensory pressure. This reflects the sensory processing difficulties reported by TBI survivors (Lew et al., 2006).

The **Cognitive Fatigue** mechanic reduces the player's movement speed proportionally as overload increases (from 100% speed at 0 overload to 55% speed at maximum). This represents how cognitive fatigue affects processing speed and motor function after TBI (Johansson et al., 2009).

2.5 Player Character and Emotional Feedback

The player character is drawn with layered ellipses (body, head, shadow) and has four distinct facial expression stages that change based on the overload level:

1. **Neutral** (overload 0-34): Straight-line mouth
2. **Anxious** (overload 35-69): Small "o" mouth with subtle eyebrow tilt
3. **Stressed** (overload 70-89): Inverted arc frown with stronger eyebrows
4. **Overloaded** (overload 90-100): Clenched teeth with vertical tooth lines and sharp eyebrows

These expressions provide an emotional visual indicator of the character's internal state without requiring the player to check the overload meter.

2.6 Sound Design

All audio is generated procedurally using the Web Audio API. This approach was chosen because it requires no external audio files and allows dynamic audio behavior.

The ambient drone is the most significant audio feature. It uses a sine wave oscillator whose frequency scales from 60 Hz to 175 Hz and whose volume scales from 0.006 to 0.055 based on the current overload level. This creates an auditory representation of sensory pressure that intensifies as the player becomes more overloaded and decreases when recovering in the Calm Zone.

Additional sound effects provide feedback for game events: a three-note ascending chime when collecting stars, a triangle wave sound when recalling the objective, a two-note notification for checkpoints, a low sawtooth warning pulse when overload exceeds 80%, a soft sine tone when recovering in the Calm Zone, and ascending/descending melodies for win and lose states.

Audio is initialized on the first user interaction (key press or mouse click) to comply with browser autoplay policies.

2.7 Level Design and Player Progression

The level consists of three rooms separated by walls with gap openings. The rooms are laid out left-to-right, with the player starting in Room 1.

Room 1 (Tutorial Phase): An open space with two stars placed nearby. The overload mechanic is inactive. Players learn movement, wall collision, and star collection without any pressure.

Room 2 (Memory Phase): After collecting the first two stars, the Memory Fade mechanic activates and the player reaches a checkpoint. One star is placed in this room. A horizontal wall obstacle creates navigation interest. A stimulus zone in the passage between Room 1 and Room 2 introduces the concept of areas that increase overload.

Room 3 (Combined Phase): Both mechanics are fully active. Two stars are placed here. The Calm Zone is positioned in the upper area, and additional stimulus zones and wall obstacles create navigation challenges. Players must balance collecting the final stars with managing overload and finding the Calm Zone.

Decorative elements (rectangles representing furniture-like objects) are placed throughout the rooms to add environmental depth. A floor tile grid provides spatial orientation in normal mode.

2.8 Low Sensory Mode

Low Sensory Mode (toggled with the L key) was implemented following game accessibility guidelines (Game Accessibility Guidelines, 2012). It dramatically changes the visual presentation:

- Floor tile grid is hidden
- Stimulus zones use outlines instead of filled pulsing overlays
- Walls are flat solid rectangles (no shadow or highlight)
- Decorations are simplified to flat solid rectangles (no shadow or highlight)
- Stars are simple solid circles (no glow animation or star polygon shape)

- Calm Zone is a flat solid rectangle (no glow, pulse, or inner pattern)
- Particles are hidden on star collection
- Screen shake, tunnel vision vignette, and overload haze are all disabled
- Background color is slightly lighter for reduced contrast
- A persistent green "LOW SENSORY MODE" badge appears on screen
- Start screen text pulse animation is replaced with static text

2.9 Checkpoint and Respawn System

A checkpoint and respawn system was added to prevent frustration for first-time players. The game provides 3 lives (respawns). When overload reaches maximum and the player has lives remaining, they respawn at their last checkpoint with overload reduced to 50 (not zero, to maintain challenge). The objective is briefly shown again after respawn to reduce confusion. Checkpoints are set automatically when the player enters a new room/phase.

2.10 Playtesting Observations

During informal playtesting sessions, several usability issues were identified:

1. Some players did not immediately notice the Calm Zone or understand that it reduced the overload meter. The Calm Zone was visually emphasized with a distinct teal color, label, glow, and border. A "Recovering..." message appears when the player is inside it.

2. Several players forgot the objective after the Memory Fade mechanic removed the text. Although the M key allowed recall, some players did not notice the instruction initially. Phase-specific hint messages were added to the HUD (e.g., "Objective fading... Press M to recall it.").

3. Players occasionally lost the game quickly because the overload meter increased faster than expected. The checkpoint/respawn system was added, and overload rates were tuned per phase (0 in tutorial, 0.02 in memory phase, 0.065 in combined phase).

---

3. GenAI Use Documentation

3.1 Task: Project Structure Setup and Core Game Loop

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 3, 2026
Task Description	Setting up the project folder structure, creating all required files (index.html, sketch.js, style.css, jsconfig.json, README.md), downloading the p5.js library, and implementing the core game state machine with start, play, win, and lose screens.
How GenAI Was Used	Claude Code was prompted with the assignment requirements (A2 PDF) and asked to scaffold the project. It created the directory structure, downloaded p5.min.js, wrote the boilerplate HTML/CSS/JS files, and built a game loop with four states and stub functions for game logic.

Field	Details
What Was Modified After Generation	The p5.js script source path in index.html was cleared (set to empty string) for manual configuration.

3.2 Task: Git Repository Setup Assistance

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 3, 2026
Task Description	Troubleshooting git push errors when pushing to GitHub.
How GenAI Was Used	Claude Code identified that the local branch was named "master" while the push target was "main", and recommended renaming the branch with `git branch -M main` before pushing.
What Was Modified After Generation	No modifications -- commands were run as suggested.

3.3 Task: Gameplay Mechanics Implementation

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 3, 2026
Task Description	Designing and implementing gameplay mechanics representing aspects of Traumatic Brain Injury, including the Memory Fade mechanic and the Sensory Overload system with a Calm Zone recovery area.
How GenAI Was Used	ChatGPT was used to brainstorm simple ways to translate TBI-related experiences into gameplay mechanics using beginner-level p5.js code. It suggested using a timer to hide the objective text and creating an overload meter that increases over time.
What Was Modified After Generation	The code suggestions were simplified to match the level of programming expected in the course. Complex math functions were removed and the mechanics were integrated into the existing game loop structure.

3.4 Task: Integrating Mechanics into the Existing Game Structure

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 3, 2026
Task Description	Integrating the gameplay mechanics into the existing game structure that uses start, play, win, and lose states.
How GenAI Was Used	ChatGPT helped identify where different parts of the mechanics should be placed in the code structure. Variables were placed in the game variables section, initialization logic in `resetGame()`, updates in `updateGame()`, and rendering in `drawPlayScreen()`.
What Was Modified After Generation	The structure was adapted slightly to fit the code that had already been created earlier in the project by another group member.

3.5 Task: Level Design and Gameplay Balance

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 4, 2026
Task Description	Designing the level structure and improving gameplay clarity.
How GenAI Was Used	ChatGPT was used to brainstorm ways to structure the level so that players learn the mechanics gradually before experiencing the full challenge. It also suggested adding checkpoints and clearer prompts to help first-time players complete the level.
What Was Modified After Generation	The level layout and star positions were adjusted manually to better match the game's mechanics and playtesting feedback.

3.6 Task: UX Design for People with TBI

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5.2)
Date	March 4, 2026
Task Description	Researching necessary features for accessibility when designing for users with Traumatic Brain Injury.
How GenAI Was Used	ChatGPT was used to help understand what aspects game designers need to know when designing for users with Traumatic Brain Injury.

Field	Details
What Was Modified After Generation	The existing simple UI instruction was modified into a more accessibility-friendly design.

3.7 Task: Full Game Enhancement (Walls, Sound, Effects, References)

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 5, 2026
Task Description	Comprehensive game enhancement to satisfy all rubric criteria. Adding wall obstacles and room layout, stimulus zones, Web Audio API sound system (collection chimes, ambient drone, overload warnings, win/lose melodies), particle effects, cognitive fatigue mechanic, tunnel vision vignette, screen shake, floor tiles, decorations, narrative framing on start/win/lose screens, ACM references as code comments, completing README sections, and Low Sensory Mode.
How GenAI Was Used	Claude Code was provided the A2 rubric PDF and asked to evaluate the existing game code against all criteria, then implement all missing features. It rewrote sketch.js with the full feature set, updated README.md with all required sections and ACM references, updated index.html with meta tags and accessibility attributes, and created the assets directory structure.
What Was Modified After Generation	Low Sensory Mode differences were further enhanced through additional prompts to make the visual distinction more dramatic (decorations changed from hidden to simplified solid rectangles, additional visual simplifications added).

3.8 Task: Player Character Facial Expressions

Field	Details
GenAI Tool Used	ChatGPT (OpenAI GPT-5)
Date	March 5, 2026
Task Description	Fixing the player character's facial expression (which appeared to smile during high overload instead of showing distress) and implementing a four-stage expression system (neutral, anxious, stressed, overloaded).
How GenAI Was Used	ChatGPT identified that the p5.js `arc()` function with angles 0 to PI draws a smile shape, and recommended flipping to PI to TWO_PI for a frown. It then provided a four-stage expression block using `line()`, `ellipse()`, `arc()`, and `rect()` for the mouth, and angled lines for eyebrows. It also helped with positioning adjustments (moving mouth up, moving eyebrows down).

Field	Details
What Was Modified After Generation	The eyebrow positions and mouth positions were manually adjusted pixel by pixel using the offset technique suggested by ChatGPT.

3.9 Task: Input Fix (M and L Keys)

Field	Details
GenAI Tool Used	Claude Code (Claude Opus 4.6) via VS Code extension
Date	March 5, 2026
Task Description	Fixing M and L key inputs that were not responding during gameplay.
How GenAI Was Used	Claude Code identified that the p5.js `key` variable can be unreliable when used alongside `keyCode`, and replaced the string comparisons (`key === "m"`) with numeric keyCode checks (`keyCode === 77` for M, `keyCode === 76` for L).
What Was Modified After Generation	No modifications -- the fix was applied as suggested.

4. Conversation Logs

4.1 Claude Code -- Project Setup (Jimin, March 3)

User (Jimin):

> Please read this document [A2 assignment PDF]. My role is to set up the project structure and core game loop (start/play/win/lose). The assigned disability for our group is Traumatic Brain Injury. Please set up the appropriate files into this directory.

Claude Code:

> Created the full project structure including: `index.html`, `sketch.js` (with 4-state game loop), `style.css`, `jsconfig.json`, `README.md`, `assets/` subdirectories (images, sounds, fonts), and `libraries/p5.min.js`.

4.2 ChatGPT -- Mechanics Implementation (Kevin, March 3)

User (Kevin):

> [A2 assignment PDF] I have a p5.js game skeleton with start, play, win, and lose states. I need to implement gameplay mechanics that represent aspects of Traumatic Brain Injury, but the code needs

> to remain simple because this is an introductory programming course.

ChatGPT:

> Suggested implementing a "Memory Fade" mechanic using a timer that hides the objective text after a short period of time, along with a key press that allows the player to recall the objective temporarily.

User (Kevin):

> I also want to represent sensory overload in the game without using complex visual effects.

ChatGPT:

> Recommended creating an overload meter that increases over time and placing a calm zone in the level where the overload value decreases when the player enters the area.

User (Kevin):

> The code should avoid advanced math functions.

ChatGPT:

> Provided simplified logic using basic variables, timers, conditional statements, and distance checks to implement the mechanics using beginner-level p5.js code.

4.3 ChatGPT -- Level Design (Catarina, March 4)

User (Catarina):

> Our group already implemented the mechanics for a p5.js game about Traumatic Brain Injury. My role is the Level Designer. I need to design a level that teaches the mechanics first and then combines them so that first-time players can understand how the game works. What level structure would work for this?

ChatGPT:

> Suggested dividing the level into three phases. Phase 1 introduces the objective and basic movement by placing a few stars close to the player so they can learn the goal without pressure. Phase 2 introduces the Memory Fade mechanic, where the objective disappears after a short time and players can press a key to recall it. Phase 3 combines both mechanics by placing the remaining stars farther away while the overload meter continues increasing.

User (Catarina):

> I also need to make sure players do not get stuck or lose too quickly during playtesting. What design changes could help make the level easier to understand?

ChatGPT:

> Recommended adding visual cues and recovery systems to help first-time players.

4.4 ChatGPT -- UX Research (Annora, March 4)

User (Annora):

> what are some aspects to know when designing for a user with Trauma Brain Injury?

ChatGPT:

> Explained the common symptoms that a TBI user might encounter, and suggested to make the experience calm, predictable and easy to resume. Listed some aspects to consider: Reduce cognitive load, Support memory and "resuming", Minimize sensory overload, Make timing forgiving, Clear feedback and error recovery, Consistency and predictability, Accessibility beyond cognition (often relevant with TBI), Give control to the user.

4.5 Claude Code -- Full Game Enhancement (Jimin, March 5)

User (Jimin):

> Based on this A2 file, can you tell me does this code game satisfies the rubric fully? Our group topic is Traumatic Brain Injury.

Claude Code:

> Evaluated the game against all 8 rubric criteria and identified critical gaps: no sound/audio, no references or citations in code or README, process doc needs PDF format, README sections incomplete, no visual assets (basic shapes only), level feels empty/minimal. Provided a detailed breakdown by criterion with specific issues and priorities.

User (Jimin):

> Then can you change all the features first to solve all of the issues and satisfies all of the rubric and requirements please. Also add sound and assets like images?

Claude Code:

> Rewrote sketch.js with comprehensive enhancements: wall obstacles creating three connected rooms, stimulus zones, Web Audio API sound system (7 sound types + dynamic ambient drone), particle effects, cognitive fatigue speed reduction, tunnel vision vignette, screen shake, floor tiles, decorations, narrative framing on all screens, Low Sensory Mode, and ACM references as code comments. Updated README.md with all required sections. Updated index.html with meta tags and accessibility attributes. Created assets directory structure.

User (Jimin):

> The M and L keyboards are not working. I mean when I pressed M or L, nothing happened.

Claude Code:

> Identified that p5.js's `key` variable is unreliable when used alongside `keyCode`. Fixed by replacing `key === "m"` with `keyCode === 77` and `key === "l"` with `keyCode === 76`.

User (Jimin):

> Make more difference between On mode of the Low sensory Mode and Off mode.

Claude Code:

> Enhanced Low Sensory Mode with dramatic visual differences: lighter background, hidden floor tiles, outlined-only stimulus zones, flat solid walls, hidden particles, simple circle stars without glow, simplified calm zone, persistent on-screen indicator badge.

User (Jimin):

> For the decorations, when the low sensory mode, do not make it as hidden entirely just make it simple solid rectangle please.

Claude Code:

> Changed decorations in Low Sensory Mode from hidden entirely to simple solid rectangles without shadow or highlight effects.

4.6 ChatGPT -- Player Facial Expressions (Annora, March 5)

User (Annora):

> this is the sketch code, and when the character in the game overload gets high, suddenly character in the game smile. why?

ChatGPT:

> In your code, the "smile" happens because of this line: `arc(px, py - 5, 6, 4, 0, PI);`. In p5.js, `arc(x, y, w, h, start, stop)` draws part of an oval between two angles. Drawing from 0 to PI often creates the bottom half of an arc, which looks like a smiling mouth. To make it look like a frown instead, flip the arc angles: `arc(px, py - 5, 6, 4, PI, TWO_PI);`

User (Annora):

> please change the version of faces in steps of neutral -> anxious -> overloaded.

ChatGPT:

> Provided a multi-stage expression system to paste inside `drawPlayer()`, replacing the old single `if (overload > 70)` block with four expression stages using combinations of `line()`, `ellipse()`, `arc()`, and `rect()` for the mouth, and angled lines for eyebrows.

User (Annora):

> `line(px - 3, py - 3, px + 3, py - 3);` how to move the mouth up

ChatGPT:

> The mouth's vertical position is controlled by the `py - 3` part. To move the mouth up, make the y-value smaller (subtract more from py). Recommended using a variable: `let mouthY = py - 6;` so it's easy to adjust.

User (Annora):

> what is the syntax of line

ChatGPT:

> In p5.js, the syntax for `line()` is: `line(x1, y1, x2, y2);` where (x1, y1) is the starting point and (x2, y2) is the ending point.

User (Annora):

> move the eyebrows down a little bit

ChatGPT:

> To move the eyebrows down, increase the y-values (bigger y = lower on screen). Suggested changing `py - 14.8` to `py - 12.8` and `py - 16.2` to `py - 14.2`. Recommended using an offset variable: `let browDown = 2;`

User (Annora):

> move the mouth and teeth up a little bit

ChatGPT:

> To move the mouth + teeth up, decrease the y-values. Suggested using one offset variable: `let mouthUp = 2;` applied to all mouth and teeth coordinates.